universität wien

MULTIOMICS DATA SCIENCE - PANDAS

TABLES

- One of the most common forms of storing big datasets are tables
- Structured in rows and columns, therefore two dimensional
- Most used tools to manipulate tables are Microsoft Excel or LibreOffice Spreadsheet

PANDAS

- Python library to open, manipulate and store tables
- Powerful tool with many functions to operate on date
- Uses NumPy library in the background
- Young library with constantly growing fan/user base

LOAD / READ-IN TABLES

- Pandas supports tables in *.xls, *.xlsx, *.csv and *.tsv format
- For *.xls, *.xlsx files, we need the filename and the working sheet name
- For *.csv and *.tsv files, we need the separation sign and optional the quote sign

```
import pandas as pd
import numpy as np   # library for vector / matrix computation

df1 = pd.read_excel("C:\path\to\file.xls", "Worksheet")
df2 = pd.read_csv("C:\path\file.csv", sep = ',', quotechar = '"')
df3 = pd.DataFrame()     # empty data frame
```

LOAD / READ-IN TABLES

- It is also possible to load tables directly from the internet
- Example on the IRIS dataset:

```
import pandas as pd
iris = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-
data/master/iris.csv') # known flower (iris) dataset
```

LOAD / READ-IN TABLES

- It is also possible to load tables directly from the internet

- Example on the IRIS dataset:

```
import pandas as pd
iris = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-
data/master/iris.csv') # known flower (iris) dataset
print(iris)
     sepal_length  sepal_width  petal_length  petal_width    species
0             5.1          3.5           1.4          0.2     setosa
1             4.9          3.0           1.4          0.2     setosa
..            ...          ...           ...          ...        ...
148           6.2          3.4           5.4          2.3  virginica
149           5.9          3.0           5.1          1.8  virginica
```

MANY OPTIONAL READ-IN PARAMETERS

- We can set many optional parameters when loading a table:
  - `index_col = ‘column name‘`
  - `na_values = [‘filtered‘, ‘div !0‘]`
  - `header = None`
  - `dtype = {‘col a‘: np.float64, ‘col b‘: np.int64, ‘col c‘: object}`
  - `skip_rows = 3`
  - …

  - Documentation: *https://pandas.pydata.org*

STORING TABLES

- It is also very uncomplicated to store dataframes in with pandas:

```
df1.to_excel("C:\path\file.xlsx", sheet_name = "My Data")
df2.to_csv("C:\path\table.csv", index = False)
```

DATAFRAMES IN PANDAS

- The internal structure of Pandas is realized with dataframes
- dataframes provide data row based and column based
- Rows and columns are indexed

DATAFRAMES IN PANDAS

- The internal structure of Pandas is realized with dataframes
- dataframes provide data row based and column based
- Rows and columns are indexed
- Column indexes are header names (default)
- Row indexes are incermenting numbers (default)

DATA ACCESS

- Pandas has two types of access, one with the known bracked [ ] operators and one with either the `loc` or `iloc` function

- Bracket operators work:
  - Column based with lists
  - Row based with boolean lists

```
iris[["sepal_length", "species"]]     # column based
iris[iris["sepal_length"] >= 5.0]     # row based
```

- The bracket operator creates when using a copy of the dataframe, therefore access is possible but no manipulation on the original dataframe

DATA ACCESS WITH LOC

- With the `loc` function, we can access rows, columns or whole blocks
- `loc` works with labels or with boolean lists (e.g., `db["width"] > 0`)
- Caution: numbers are interpret as label index, nota position index

		0	1	2
		size	*width*	*11*
0	1	a	1	foo
1	5	a	2	bar
2	6	e	2	blub

position index

label index / hearder row

label index / index column

position index

universität wien

DATA ACCESS WITH LOC

- `loc` accesses are row first, column second oriented

- Example:

```
df.loc[1, 'width']
df.loc[df['size'] == 'a', 11]
df.loc[[5, 6]]
df.loc[:, ['size']]
```

		0	*1*	*2*
		size	*width*	*11*
0	1	a	1	foo
1	5	a	2	bar
2	6	e	2	blub

universität wien

DATA ACCESS WITH LOC

- `loc` accesses are row first, column second oriented

- Example:

```
df.loc[1, 'width']
df.loc[df['size'] == 'a', 11]
df.loc[[5, 6]]
df.loc[:, ['size']]
```

		0	1	2
		size	*width*	*11*
0	1	a	1	foo
1	5	a	2	bar
2	6	e	2	blub

universität wien

DATA ACCESS WITH LOC

- `loc` accesses are row first, column second oriented

- Example:

```
df.loc[1, 'width']
df.loc[df['size'] == 'a', 11]
df.loc[[5, 6]]
df.loc[:, ['size']]
```

		0	1	2
		size	*width*	*11*
0	1	a	1	foo
1	5	a	2	bar
2	6	e	2	blub

universität wien

DATA ACCESS WITH LOC

- `loc` accesses are row first, column second oriented

- Example:

```
df.loc[1, 'width']
df.loc[df['size'] == 'a', 11]
df.loc[[5, 6]]
df.loc[:, ['size']]
```

		0	1	2
		size	*width*	*11*
0	1	a	1	foo
1	5	a	2	bar
2	6	e	2	blub

universität wien

DATA ACCESS WITH LOC

- `loc` accesses are row first, column second oriented

- Example:

```
df.loc[1, 'width']
df.loc[df['size'] == 'a', 11]
df.loc[[5, 6]]
df.loc[:, ['size']]
```

		0	1	2
		size	*width*	*11*
0	1	a	1	foo
1	5	a	2	bar
2	6	e	2	blub

universität wien

ASSIGNMENT WITH LOC

- When assigned data is not scalar, the dimensions of the loc section must fit with each othern

- Exemplary assignment:

```
df.loc[1, 'width'] = 1
df.loc[df['size'] == 'a', 11] = df2.loc[df['size'] == 'a', "width"]
df.loc[:, ['size']] = df3     # dimensions must fit

print(df.shape, df3.shape)    # getting data frame dimensions
```

DATA ACCESS WITH ILOC

- `iloc` is based on position indexes
- Exemplary access:

```
df.iloc[0, 1]                        # df.loc[1, 'width']
df.iloc[df['size'] == 'a', 1]        # df.loc[df['size'] == 'a', 'width']
df.iloc[[1, 2]]                      # df.loc[[5, 6]]
df.iloc[:, 0]                        # df.loc[:, ['size']]
```

		0	*1*	*2*
		size	*width*	*11*
0	1	a	1	foo
1	5	a	2	bar
2	6	e	2	blub

universität wien

DATA ACCESS WITH ILOC

- `iloc` is based on position indexes
- Exemplary access:

```
df.iloc[0, 1]                          # df.loc[1, 'width']
df.iloc[df['size'] == 'a', 1]          # df.loc[df['size'] == 'a', 'width']
df.iloc[[1, 2]]                        # df.loc[[5, 6]]
df.iloc[:, 0]                          # df.loc[:, ['size']]
```

		0	1	2
		size	*width*	*11*
0	1	a	1	foo
1	5	a	2	bar
2	6	e	2	blub

universität wien

DATA ACCESS WITH ILOC

- `iloc` is based on position indexes
- Exemplary access:

```
df.iloc[0, 1]                          # df.loc[1, 'width']
df.iloc[df['size'] == 'a', 1]          # df.loc[df['size'] == 'a', 'width']
df.iloc[[1, 2]]                        # df.loc[[5, 6]]
df.iloc[:, 0]                          # df.loc[:, ['size']]
```

		0	1	2
		size	*width*	*11*
0	1	a	1	foo
1	5	a	2	bar
2	6	e	2	blub

universität wien

DATA ACCESS WITH ILOC

- `iloc` is based on position indexes
- Exemplary access:

```
df.iloc[0, 1]                          # df.loc[1, 'width']
df.iloc[df['size'] == 'a', 1]          # df.loc[df['size'] == 'a', 'width']
df.iloc[[1, 2]]                        # df.loc[[5, 6]]
df.iloc[:, 0]                          # df.loc[:, ['size']]
```

		0	1	2
		size	*width*	*11*
0	1	a	1	foo
1	5	a	2	bar
2	6	e	2	blub

universität wien

DATA ACCESS WITH ILOC

- `iloc` is based on position indexes
- Exemplary access:

```
df.iloc[0, 1]                          # df.loc[1, 'width']
df.iloc[df['size'] == 'a', 1]          # df.loc[df['size'] == 'a', 'width']
df.iloc[[1, 2]]                        # df.loc[[5, 6]]
df.iloc[:, 0]                          # df.loc[:, ['size']]
```

		0	1	2
		size	*width*	*11*
0	1	a	1	foo
1	5	a	2	bar
2	6	e	2	blub

universität wien

DATA ACCESS WITH LOC AND ILOC

- Caution:

```
df.iloc[0]            # Series (vector) with |column| entries
df.iloc[[0]]          # dataframe with one row

df.loc[:, "size"]     # Series (vector) with |row| entries
df.loc[:, ["size"]]   # dataframe with one column
```

- Different data types → different functions

ADDING / DELETING COLUMNS

```
df = pd.DataFrame()  # Creating empty dataframe
# When adding first column, size of
# dataframe will be defined
df["numbers"] = [1, 2, 3, 4]
df["text"] = ["I", "am", "a", "dataframe"]

del df["numbers"]
```

ADDING / DELETING ROWS

```
# adding
df.loc[len(df)] = ['b', 3, 'a word']
df.append({'size': 'c', 11: 5, 'width': 'bam'}, ignore_index = True)

# Caution: when deleting rows, the label index
# of the remaining rows remains unchanged
df = df.iloc[[0,2,4]]
```

COLUMN/ROW-BASED AGGREGATION

```
col_s = example.sum()            # column-based sum
# length       7
# width       15
# height      18

row_s = example.sum(axis = 1)   # row-based sum
# 0      3
# 1      8
# 2      3
# 3      8
# 4      4
# 5      7
# 6      1
# 7      6
```

example:

length	*width*	*height*
3	0	0
3	1	4
0	0	3
0	4	4
1	2	1
0	3	4
0	1	0
0	4	2

KNOWN FUNCTIONS ON DATAFRAMES

```
iris.mean()                # column-based mean values
iris.mean(axis = 1)        # row-based mean values

iris.isna()                # cell-based test on NaN values
iris.notna()               # cell-based test on true values

iris.isnull()              # cell-based test on missing values
iris.notnull()             # cell-based test on non-missing values

iris.max()                 # column-based maximum value
iris.max(axis = 1)         # row-based maximum value
```

KNOWN OPERATIONS ON DATAFRAMES

```
iris2 = iris + 5          # iris + 5 creates new dataframe
iris *= 5
iris **= 2
iris2 = iris / 3
iris2 -= 10
```

COLUMN GROUPED MEAN VALUES

- We need to know the grouping of column names in order to compute mean values:

```
mean_names = {"length": ["sepal_length", "petal_length"],
              "width": ["sepal_width", "petal_width"]}

iris_means = pd.DataFrame()
for mean_name, column_names in mean_names.items():
    iris_means[mean_name] = iris[column_names].mean(axis = 1)
```

COLUMN GROUPED MEAN VALUES

- We need to know the grouping of column names in order to compute mean values:

```
mean_names = {"length": ["sepal_length", "petal_length"],
              "width": ["sepal_width", "petal_width"]}

iris_means = pd.DataFrame()
for mean_name, column_names in mean_names.items():
    iris_means[mean_name] = iris[column_names].mean(axis = 1)
```

sepal_ length	sepal_ width	petal_ length	petal_ width	species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa

→

length	width
3.25	1.85
3.15	1.6

universität wien

CASE STATEMENTS

- We want to create a new column dependent on an existing column:

```
print(df)
    person   points
0   hugo     82
1   micha    22
2   lisa     95
3   dieter   78
4   uschi    40
5   bob      5
```

CASE STATEMENTS

```
import numpy as np
df[‘grade‘] = np.where(df['points'] < 60, 5,
             np.where(df['points'] < 70, 4,
             np.where(df['points'] < 80, 3,
             np.where(df['points'] < 90, 2, 1))))
print(df)
    person   points   grade
0   hugo     82       2
1   micha    22       5
2   lisa     95       1
3   dieter   78       3
4   uschi    40       5
5   bob      5        5
```

GROUPING

```
print(table)
    article  category     price
0   ...      book         20.00
1   ...      game         95.99
2   ...      book         15.00
3   ...      telephone    299.00
4   ...      book         22.00
5   ...      telephone    699.95

table[['category', 'price']].groupby(['category']).sum()
category       price
book           57.00
game           95.99
telephone     998.95
```

GROUPING

```
print(table)
    article  category     price
0   ...      book         20.00
1   ...      game         95.99
2   ...      book         15.00
3   ...      telephone    299.00
4   ...      book         22.00
5   ...      telephone    699.95

table[['category', 'price']].groupby(['category']).mean()
category       price
book           19.00
game           95.99
telephone     499.475
```

universität wien

GROUPING

```
print(table)
    article  category     price
0   ...      book         20.00
1   ...      game         95.99
2   ...      book         15.00
3   ...      telephone    299.00
4   ...      book         22.00
5   ...      telephone    699.95

table[['category', 'price']].groupby(['category']).max()
category       price
book           22.00
game           95.99
telephone     699.95
```

universität wien

MERGE FUNCTION

```
df1 = pd.DataFrame({'lkey': ['foo', 'bar', 'baz', 'foo', 'bum'],
                    'value': [1, 2, 3, 5, 6]})
df2 = pd.DataFrame({'rkey': ['foo', 'bar', 'baz', 'foo', 'bam'],
                    'value': [5, 6, 7, 8, 9]})

df1                          df2
    lkey value                   rkey value
0   foo      1               0   foo      5
1   bar      2               1   bar      6
2   baz      3               2   baz      7
3   foo      5               3   foo      8
3   bum      6               4   bam      9
```

MERGE FUNCTION

```
df_merge = df1.merge(df2, left_on='lkey', right_on='rkey')

df_merge
  lkey  value_x rkey  value_y
0  foo        1  foo        5
1  foo        1  foo        8
2  foo        5  foo        5
3  foo        5  foo        8
4  bar        2  bar        6
5  baz        3  baz        7
```

universität wien

MERGE FUNCTION

```
df_merge = df1.merge(df2, left_on='lkey', right_on='rkey', how='left')

df_merge
  lkey  value_x rkey  value_y
0  foo        1  foo        5
1  foo        1  foo        8
2  foo        5  foo        5
3  foo        5  foo        8
4  bar        2  bar        6
5  baz        3  baz        7
6  bum        6  NaN      NaN
```

universität wien

MERGE FUNCTION

```
df_merge = df1.merge(df2, left_on='lkey', right_on='rkey', how='right')

df_merge
  lkey  value_x rkey  value_y
0  foo        1  foo        5
1  foo        1  foo        8
2  foo        5  foo        5
3  foo        5  foo        8
4  bar        2  bar        6
5  baz        3  baz        7
6  NaN      NaN  bam        9
```

universität wien

MERGE FUNCTION

```
df_merge = df1.merge(df2, left_on='lkey', right_on='rkey', how='outer')

df_merge
  lkey  value_x rkey  value_y
0  foo        1  foo        5
1  foo        1  foo        8
2  foo        5  foo        5
3  foo        5  foo        8
4  bar        2  bar        6
5  baz        3  baz        7
6  bum        6  NaN      NaN
7  NaN      NaN  bam        9
```

universität wien

CONCAT FUNCTION

```
df_concat = pd.concat([df1, df2])

df_concat
  lkey  value rkey
0  foo      1  NaN
1  bar      2  NaN
2  baz      3  NaN
3  foo      5  NaN
4  bum      6  NaN
0  NaN      5  foo
1  NaN      6  bar
2  NaN      7  baz
3  NaN      8  foo
4  NaN      9  bam
```

universität wien

CONCAT FUNCTION

```
df_concat = pd.concat([df1, df2])

df_concat                    # label index in rows unchanged
  lkey  value rkey
0  foo      1  NaN
1  bar      2  NaN
2  baz      3  NaN
3  foo      5  NaN
4  bum      6  NaN
0  NaN      5  foo
1  NaN      6  bar
2  NaN      7  baz
3  NaN      8  foo
4  NaN      9  bam
```

universität wien

PIVOT TABLES

- There are mainly two common table formats:
  - Pivot tables with *m* features x *n* entries

	M1	*M2*	*M3*
E1	10	1	5
E2	42	71	9

PIVOT TABLES

- There are mainly two common table formats:
  - Pivot tables with *m* features x *n* entries
  - Long tables, features and entries are in two separate columns

	M1	*M2*	*M3*
E1	10	1	5
E2	42	71	9

	Entry	*Feature*	*Value*
0	E1	M1	10
1	E1	M2	1
2	E1	M3	5
3	E2	M1	42
4	E2	M2	71
5	E2	M3	9

PIVOT TABLES

- There are mainly two common table formats:
  - Pivot tables with *m* features x *n* entries
  - Long tables, features and entries are in two separate columns

	M1	*M2*	*M3*
E1	10	1	5
E2	42	71	9

- In long tables, entry/feature pairs e.g. (E1, M1) may occur several times

	Entry	*Feature*	*Value*
0	E1	M1	6
1	E1	M1	4
2	E1	M2	1
3	E1	M3	5
4	E2	M1	42
5	E2	M2	71
6	E2	M3	9

PIVOT TABLES

- Command to transform long tables in pivot tables:

```
df = pd.DataFrame({"Entry": ["E1", "E1", "E1", "E1", "E2", "E2", "E2"],
                   "Feature": ["M1", "M1", "M2", "M3", "M1", "M2", "M3"],
                     "Value": [6, 4, 1, 5, 42, 71, 9]})
df_pivot = df.pivot_table(index = "Entry", columns = "Feature",
                          values = "Value", aggfunc = sum)
```

	M1	*M2*	*M3*
E1	10	1	5
E2	42	71	9

	Entry	*Feature*	*Value*
0	E1	M1	6
1	E1	M1	4
2	E1	M2	1
3	E1	M3	5
4	E2	M1	42
5	E2	M2	71
6	E2	M3	9

universität wien

PIVOT TABLES

- Command to transform long tables in pivot tables:

```
df = pd.DataFrame({"Entry": ["E1", "E1", "E1", "E1", "E2", "E2", "E2"],
                   "Feature": ["M1", "M1", "M2", "M3", "M1", "M2", "M3"],
                     "Value": [6, 4, 1, 5, 42, 71, 9]})
df_pivot = df.pivot_table(index = "Entry", columns = "Feature",
                          values = "Value", aggfunc = sum)
```

	M1	*M2*	*M3*
E1	10	1	5
E2	42	71	9

	Entry	*Feature*	*Value*
0	E1	M1	6
1	E1	M1	4
2	E1	M2	1
3	E1	M3	5
4	E2	M1	42
5	E2	M2	71
6	E2	M3	9

- Other *aggregate* functions:
  `np.mean, np.max, np.min`

universität wien

Source: *https://img-9gag-fun.9cache.com/photo/aVv76ev_460swp.webp*